Serial Protocol Decode — measured reference

Ian Ameline · version 1.10 · MIT licence

The detail behind MANUAL.md. Everything here was measured on a DMM6500 (firmware 1.7.17a) against an SDG2122X generator, verified where noted with an SDS1204X-E scope. The manual is written for someone using the app; this file is for someone who needs the numbers or is changing the code.

Every number in this file is a DMM6500 number. The app is expected to run on the wider Keithley TSP family — the README's **Which instruments** section sets out which models and on what grounds — but nothing here has been reproduced on any of them, and several of these figures are ones you would expect to move. The sample-rate ceiling, the press latencies, the display-object budget and the reading-buffer throughput are all properties of this instrument's hardware and firmware, not of the decoder. Treat them as measurements of one machine until someone repeats them.

The figures split into two classes, and only one of them should transfer. Anything descending from the **sample rate** — the rate ladder, the samples-per-bit table, the recording-length arithmetic — is timing, and a model at the same 1 MS/s inherits it unchanged. Anything descending from the **analog front end** is not: the **Tolerance envelope** below, **Levels and offsets**, the **marginal at 2400 Bd** and the **–2 V logic-low band** are all measurements of one acquisition board.

The repeating-pattern rate misfit belongs to neither class. It is arbitration between two readings of the same pulse lengths, it reproduces with no acquisition board in the path at all, and it therefore transfers to every model unchanged — including the sample-rate dependence that drives it, since the candidates a reading admits depend on how many samples per bit the capture holds.

Those are the ones to re-measure first on another model, and on a DMM7510 they could plausibly move **in the app's favour**: its acquisition boards and digitizer are a significant step up, which is exactly the axis the –2 V band and the threshold picker's histogram sensitivity live on. Better is still different. A wider envelope would be welcome; a *shifted* one would quietly invalidate the table.

On a DAQ6510 there is no such split, and every figure here should hold. It shares both the UI board and the acquisition board with the DMM6500 — only the channel-board plugin differs — so the analog numbers are measurements of the same hardware, not of comparable hardware. That makes it the one model against which this file is a **falsifiable** document rather than an approximate one: a deviation is a finding, not an expected difference. It is therefore the first port to try and the cheapest to interpret.

Press latency

A front-panel **touch** press is **not dispatched while a script is running**, and presses **queue** rather than being dropped. So a handler's own duration *is* its press latency, and returning is the only way to answer a touch press. That is why a stop press is absorbed after the capture it interrupted.

The **front-panel TRIGGER key was meant to be the exception** — latched by the trigger hardware rather than delivered to the interpreter, so a running handler could see it. Measured 2026-08-18, it is not: the press does not reach the latch while a panel-initiated run executes, so **nothing makes a minutes-long press interruptible**, and what makes it acceptable is only that the run is bounded and says its cost beforehand.

Measured over 50 button presses covering every control in every state:

Press	Measured
Anything that does not capture	**77 ms or less** (Save is the slowest of them)
Capture, rate locked	**1.7 s**
Capture, unlocked, 9600 Bd and up	3.6 – 6.5 s
Capture, unlocked, 2400 Bd	4.9 **– 9.1 s**
A recording, start to filed	**one press**, as long as the window takes — see below

The unlocked figure is a **range, not an average, and it is genuinely variable**: twelve back-to-back unlocked captures on one unchanged 2400 Bd line measured min 4.891 s, mean 5.358 s, max 7.253 s — a 2.36 s spread, 33 % of the maximum — and a separate run has been seen at 9.086 s. What varies is how many looks the rate probe needs, which depends on where the traffic is when the capture starts. `bench_panel.py` therefore bounds capture presses at **12 s**: margin sized to the measured spread rather than to a single sample, because a gate that fails at random teaches you to ignore it.

An unlocked capture costs more the **slower** the line, which is the opposite of the intuition. The rate probe starts at 1 MS/s, where 20 000 samples is 20 ms — less than one frame below about 1200 Bd — so it steps its sample rate down until it catches traffic, and each step is another capture. Locking removes the probe entirely, which is why the first press auto-locks.

That a capture exceeds 500 ms is deliberate. A 240-byte window is 19 200 samples and reading one sample out of the capture buffer costs 21.7 μs, so the read alone is 0.42 s before any decoding happens. Splitting one capture across several presses would shorten the press, but one press for one capture is worth more.

A recording press is minutes, not milliseconds, and that is now the design rather than a compromise. One press records a window, decodes all of it and files it: at 9600 baud that is 34 s of recording plus ~110 s of decoding for 32 kB, or 8.5 s plus ~30 s for 8 kB. Decoding runs at ~300 byte/s end to end and does not scale with the baud rate. `bench_panel.py` bounds such a press at **300 s**, which is a hang detector rather than a latency budget — the duration is the window size the operator chose, and nothing shortens it.

Sample rates and samples per bit

FRAME mode and the streaming modes pick their sample rate differently, and it matters:

- **FRAME** uses `fs_for_baud`, which oversamples from a discrete rate ladder, so the ratio is not constant: measured 8.33 samples/bit from 300 Bd to 19200 Bd, 8.68 at 115200 Bd, and 4.00 at 250000 Bd.
- **Streaming** uses `fs_for_burst`, the *lowest* listed rate that still delivers `minsabit` = 4 samples/bit, so it asks for ~4.17 samples/bit from 300 Bd to 19200 Bd and buys twice the recording length for it. Also ladder-dependent: 5.21 at 38400 Bd and 57600 Bd, 5.56 at 115200 Bd. The *delivered* ratio is slightly lower than the requested one, because `acq_overhead` costs ~0.51 us a sample.

Tolerance follows **samples per bit, not baud rate**, and the sample-rate ladder does not give every rate the same figure — 8.3 at 9600, 8.6 at 28800, 10.0 at 1500, 4.0 at 250 000. It holds at least ~8 up to 115200 Bd. Read `SA/BIT` rather than assuming a slow line is the robust one.

Recording length, and where the limits come from

There are **two recording modes**, capped at **8 192** and **32 768 bytes**. Nothing records without a byte ceiling, and beyond the ceiling the answer is flow control rather than a bigger buffer.

The two differ only in size, at every rate, and the choice is a responsiveness trade rather than a capability: one press records a window, decodes it and files it, so `8 kB` reaches bytes in about a quarter of the time, while `32 kB` spends less of the run on per-window overhead. Both are lossless.

The capture buffer holds **2 800 000 readings** (`ck_bufmax`). A mode asks for whatever its ceiling needs at the locked rate -- `cap x framebits x (fs / baud)` -- capped by the buffer. The table below is the 32 kB window; divide the readings and the durations by four for `8 kB`. Divide by `fs_for_burst` for the signal duration:

Locked rate	Readings	Signal	Wall clock to fill
300 Bd	1 365 334	1092 s	1092 s
1200 Bd	1 365 334	273 s	273 s
2400 Bd	1 365 334	136 s	136 s
4800 Bd	1 365 334	68 s	68 s
9600 Bd	1 365 334	34.1 s	34.1 s
19200 Bd	1 365 334	17.1 s	17.1 s
38400 Bd	1 706 667	8.5 s	17.1 s
57600 Bd	1 706 667	5.7 s	17.1 s
115200 Bd	1 820 445	2.8 s	18.2 s
153600 Bd	2 133 334	2.1 s	21.3 s

Signal and wall clock diverge above ~19200 Bd for the reason in the next section. Verified against the app's own arithmetic, not hand-computed.

The highest standard rate that records is 153600 Bd, and the cut-off is exact rather than approximate: `fs_for_burst` returns nil once `1/(minsabit * baud)` no longer exceeds `acq_overhead` plus one period of the top listed rate, which is `1/(4 * (1/1e6 + 5.1e-7))` = **165563 Bd**. 153600 clears it at 4.31 samples/bit delivered; 172800 is the next entry in the ladder and does not. Confirmed on the instrument 2026-08-18 on both sides of the boundary: an 8 kB recording at 153.6 kBd captures and decodes, and at 172.8 kBd the mode is refused. The refusal names the recording's own figure: "*192000 Bd: a recording gets 3.4 samples/bit, under the 4 floor -- use FRAME*" — deliberately not the `SA/BIT` header value, which is measured from a FRAME capture and reads about 5.2 at that rate. Recording needs 4 samples/bit and the digitizer stops at 1 MS/s, so 172800 Bd and beyond -- including the 250 kBd decode ceiling -- are FRAME only.

Nothing is dropped inside a recording. One hardware acquisition with no script running, so there are no gaps to fall through. Measured at 9600 Bd: **1925 of 1925 bytes contiguous** against a non-repeating payload, at a byte rate matching the wire to 0.5 %. Re-verified at 115200 Bd with the 1024-byte non-repeating `v71` payload: 400 checked bytes were **one unbroken slice** of it. And a full 2 800 000-reading recording at 4800 Bd decoded **67 749 bytes against 67 200 expected** for 140 s of line -- a 0.8 % match, with the ASCII gutter showing continuous text straight through the payload wrap.

Beyond one window: flow control, not a bigger buffer

A window is not a total. `Options > Rear BNC = FC Out` asserts `trigger.extout` once per armed capture -- `stream_arm` (serial_app.tsp), `acq_free` and `acq_triggered` (serial_core.tsp), with probe reads suppressed via `nofc` so a level-learning probe does not spend a credit. One pulse means "send up to one window". While Lua is decoding nothing is armed, so a device that waits for the credit cannot transmit into a closed window, and the byte log appends across windows. Unlimited total, bounded throughput.

And no interaction per window. `record_run()` loops credit -> record -> decode -> file -> credit inside the one press, so the operator presses Capture once for a transmission of any length. What ends it is under **The flow-control loop's bound** below.

Measured electrically, on the 50 Ω matched cabling, with the pulse read on the same split that feeds the generator: **idle-low at +0.20 V, high +4.45 V, peak +4.82 V, 5.72 μs wide** — and the width is the same at three thresholds (5.72 μs at 50 %, 5.71 at 2.5 V, 5.73 at 2.0 V), so it is a real width rather than an artefact of where it was measured. **Rise 10–90 % is 436 ns, 7.6 % of the pulse: the corners are square.** An earlier bench observation that it "looked like the top of a sine wave" is not reproduced.

Those figures supersede an earlier **4.92 V peak, –0.96 V baseline, 9.4–9.8 μs** taken on 1 m RG179 75 Ω cabling. The width differs by 1.7× and the –0.96 V undershoot is absent, which is what a reflection on the old unterminated path would look like. The old numbers are not wrong about that bench; they do not describe this one.

The width CANNOT BE SET on this firmware: `trigger.extout.pulsewidth` reads nil on 1.7.17a, confirmed again here, so `sdec.fc_pulse` (100 μs) is never applied. **A device waiting for a credit therefore needs an edge-triggered input, an interrupt or a latch** — 5.7 μs is under 1.5 bit times at 250 kBd, and a polling loop can miss it.

Still not closed as a loop, and now for a located reason. See *Triggering, in both directions* below: the generator's burst machinery does accept the arbs, but it did not act on this pulse.

The reading buffer accepts ~100 000 readings/s, and that is wall clock, not signal

The durations above are **signal** time — buffer size ÷ the sample rate the app asks for — and they are correct. What they are not is how long you wait. Measured fill rates, polling `buf.n` from the host while a press-driven recording ran:

Baud	Mode	Asked	Achieved	Ratio
4800	32 kB	20 kS/s	19 798/s	0.99×
9600	32 kB	40 kS/s	39 200/s	0.98×
19200	32 kB	80 kS/s	76 858/s	0.96×
38400	32 kB	200 kS/s	99 996/s	0.50×
57600	32 kB	300 kS/s	99 986/s	0.33×
115200	32 kB	640 kS/s	99 994/s	0.16×
160000	32 kB	1 MS/s	100 006/s	0.10×

This is not a configuration mistake and there is nothing to fix. It was measured three ways at `samplerate` = 1 MS/s, and all three land on the same ceiling:

Configuration	Readings	Time	Rate
`digitize.count = 1`, `SimpleLoop(200000)` — what the app does	200 000	2.017 s	99 146/s
`BLOCK_MEASURE_DIGITIZE` with count = 200 000	200 000	2.119 s	94 400/s
blocking `dmm.digitize.read(buf)`, `count` = 200 000	200 000	2.186 s	91 502/s
blocking `dmm.digitize.read(buf)`, `count` = 1 000 000	1 000 000	10.930 s	91 487/s

Two things worth knowing from that table. **`SimpleLoop` takes exactly one reading per iteration and ignores `dmm.digitize.count`** — `count = 200000` with one iteration returns `n = 1`, so the loop count *is* the reading count on that path. And the blocking read, which is what FRAME mode uses and which genuinely samples at 1 MS/s, files readings at the same ~91 k/s. So the ceiling is the buffer write, not the digitizer and not the trigger model.

The digitizer really does sample at the rate it is told. At 115200 Bd the app asked for 640 kS/s and `acq_measure_fs` read back **479 677 S/s** with 4.16 samples/bit, and the bytes were contiguous and correct. The consequence is only wall clock: 505 723 readings gathered over 5.0 s of wall time represented **1.054 s of signal**, and the 12 171 bytes decoded match 1.054 s × 11 520 B/s to 0.2 %.

So above ~19200 Bd, filling the buffer takes roughly `capacity / 100 000` seconds regardless of rate — about 17-20 s for every mode in the table.

Stopping a run: there is no control that does it

A decode long enough to want stopping is a handler that has not returned, and a touch press is not delivered until it does — so the panel cannot offer a control, and the front-panel TRIGGER key does not deliver `trigger.EVENT_DISPLAY` while a panel-initiated run executes. A run therefore lasts as long as its size requires. That is why the size is chosen before the press and why the panel states the byte ceiling and shows a progress bar rather than an estimate.

The flow-control loop's bound

Under `FC Out`, one press credits the device, records a window, decodes it, files it and credits again, ending when a credited window comes back silent. A device that never stops talking would otherwise hold the panel indefinitely, so the loop has **two** backstops, and it is worth knowing why one is not enough:

bound	value	what it limits
`fc_maxwin`	32 windows	the BYTES — 1 MB at the 32 kB window size
`fc_maxsec`	1200 s (20 min)	the WALL CLOCK, decode included

A window count is not a time. Each window's acquisition is bounded independently at `min(stream_maxwait, strm_maxsec)`, and the decode costs more than the acquisition did — measured at 9600 baud, 34 s to fill a 32 kB window and about 109 s to decode it. So 32 windows is **76 minutes** there, and at 300 baud, where every window waits out the full 1200 s acquisition ceiling, the count alone allowed **ten and a half hours**. A panel that comes back the following afternoon is a power cycle in practice.

So the clock is the bound that binds: 20 minutes is roughly **eight** windows at 9600 baud, not 32. That is a deliberate reduction in bytes per press, and it costs no DATA — the sender only transmits when credited, so it is still waiting where the run stopped. What it costs is two presses and a file boundary, described below.

Both are backstops, and they are the ONLY things that end a run early — no key does. When either fires the note row names which one and gives the remedy, **from the first window onward** — a run stopped by the clock on window one is the normal case on a slow line.

The remedy is `Mode`, then `Capture`, and it continues in a NEW file. Not one press, and not the same file: every way out of a recording goes through `mode_exit()`, which sets `capmode` back to `frame` *and* nils `flog_path`. So `Capture` on its own takes a frame capture, and the bytes that follow land in the next numbered `bytesNNN.txt`. **The data is not lost** — the device is still waiting for a credit that has not gone out, so nothing was transmitted in the gap — but a bounded run is *fragmented across two files*, not truncated.

> **Open decision for a future version.** If a bounded run should continue into the same file, that means keeping `flog_path` across a `mode_exit()` that ends on a bound — which interacts with `NewLog`'s "closed *file* — the next capture starts a new file" note and with the per-capture eviction rules. That is a design change rather than a fix, so it is recorded here and not made.

These two bounds are the only exits the loop has, so they are what keeps a talkative device from holding the panel indefinitely.

The idle watchdog and remembered levels

A recording's quiet-line exit needs a voltage threshold, and `clear_result()` zeroes the measured one at the top of every capture — so `stream_begin()` probes the line first. **Under flow control that probe looks at a silent line**, because the device is waiting for its credit, and the same is true of every window after the first. Without a fallback the watchdog would be disarmed exactly where it matters most, and every window would wait out `strm_maxsec`.

So `sig_levels()` keeps the last levels it successfully measured (`lvl_thr`, `lvl_hyst`, `lvl_swing`), past `clear_result()`, and `stream_begin()` falls back to them when its own probe finds nothing. They exist whenever a rate is locked, and locking a rate is already required to enter a recording mode. With no remembered levels the watchdog stays disarmed, which is the safe answer: a truncated recording is worse than a long one.

`display.waitevent(0)` blocks and wedges the instrument. Do not use it. The reference documents it as `<object id>, <sub id> = display.waitevent([timeout])`, and the instrument's own display example polls it with a 0 timeout inside a long loop to catch a Stop press.

On firmware 1.7.17a a 0 timeout does not mean "return immediately with whatever is pending". With no waitevent-enabled object to report, it waits: the TSP interpreter stops answering, the control socket goes silent, and reconnecting does not recover it. Only a power cycle does. Confirmed both with the app running and on a bare instrument with nothing built. Recorded as `DMM_WAITEVENT_WEDGES_THE_INSTRUMENT` in `tools/instruments.py`.

A second obstacle sits behind it: an object's event slot is *either* a `waitevent` flag *or* a command string, never both (Display API reference, `setevent`), and every button in this app is a command string.

`display.OBJ_TIMER` is **not** a DMM6500 object type. It appears nowhere in this instrument's display API reference, so there is no timer object to drive work from.

Tolerance envelope

Measured at 8 samples per bit. In every case beyond the limit the failure is **reported** rather than silent, except where noted.

Condition	Byte-exact to
Clock error, rate **locked**	**±4 %** — past this, running *fast* corrupts bytes silently
Clock error, **auto-detect**	±12 % and beyond — it measures rather than assumes
Timing jitter	**±15 %** of a bit time — ±15.6 μs at 9600 Bd, ±1.3 μs at 115200. Past ±20 % it can fail silently
Amplitude noise	~40 % of the logic swing — ~1.3 V p-p on a 3.3 V line
Low-pass filtering (poor probe, long cable)	RC up to **0.7 bit times** — a ~2.2 kHz filter at 9600 Bd
Slow edges (open-drain pull-up)	rise up to ~40 % of a bit time — 42 μs at 9600 Bd, 3.5 μs at 115200
Impulse spikes / dropouts	usually costs bytes and raises errors; a spike confined to a *data* bit changes that byte undetectably
Logic swing	**1.6 V** to 5 V TTL — the limit is the swing, not the family
DC offset	anything keeping both levels inside ±10 V

The UART framing cliff itself is 0.5/9.5 = 5.26 %; `ratemargin` = 0.04 is where the app starts warning.

Levels and offsets, verified

Byte-exact from –5 V to +5 V of offset with a 3.3 V swing, and across the same offset range with a **1.6 V** swing — including 1.6 V sitting at 5.05…6.65 V and at –5.06…–3.46 V. A 60 mV swing is refused -- by the swing floor (`minswing` = 0.10 V, which reports the measured swing) or as a line with no transitions, depending on which test trips first. The useful floor is between the two.

Rates verified on hardware

43 matrix points and 19 non-standard rates, byte-exact, with no instrument event logged: six frame formats, the standard ladder 300 – 250 000 Bd, a 1 kB non-repeating payload, three logic swings and twelve DC offsets.

Non-standard rates matter because a device with an awkward crystal divisor does not sit on the standard ladder: 900, 1500, 2800, 3600, 7200, 12000 and 16000 Bd all decode byte-exact, as do arbitrary values like 1379, 2731, 5333, 8123, 13333, 21600, 29127, 41666, 53333, 71111, 89123 and 104857.

A rate within **2 %** (`snaptol`) of a standard one is *reported* as the standard one — 29127 Bd reads as 28800 — because real devices use standard rates and 2 % is far inside the framing margin. The bytes are unaffected.

Above the 250 kBd ceiling it refuses by name rather than reporting a submultiple: 255 kBd, 260 kBd and 300 kBd were each declined with the samples-per-bit reason.

100 baud is the declared floor (`sdec.minbaud`), refused by name the way the 250 kBd ceiling is, and it carries `plaustol` at both ends so a rate sitting on a limit is not rejected for a rounding error. Nothing below it is supported: `stdbaud` starts at 110 Bd, and a rate typed under 110 reads as 0, meaning auto-detect.

That floor is what rules out relay-driven links. A mechanical relay must be settled before the receiver's mid-bit sample, so switching time caps the rate at `1 / (2 · t_settle)` — about 100 Bd for a 5 ms micro-relay, putting 300 Bd out of reach threefold and 110 Bd short by a tenth. The rates that would actually fit are 75 Bd and below, which is under the floor. Relays make a poor transmitter regardless: make and break times differ, so each edge carries a systematic offset; contact bounce adds edges inside the start bit, which surfaces as rejected false starts rather than framing errors; and at ~55 operations per second a 110 Bd link spends a 10^5-cycle contact rating in half an hour. The sample-rate ladder is not the obstacle — 1 kS/s is 9 samples/bit at 110 Bd — and no figure in this document is measured below 300 Bd.

A known marginal at 2400 Bd

The 1 kB non-repeating payload at 2400 Bd (25 kS/s) has produced flagged bytes in every sweep run: 1 flagged in two runs, 2 bad interior frames in a third. **The app flags them — it has never decoded that point silently wrong.** Whether the cause is the generator's arb playback at that rate or the decoder is not yet established; the offsets need checking for repeatability. Tracked, not fixed.

Ambiguity, and what FIT does not tell you

`FIT` **is not a probability that the rate is right.** It measures how well one bit period explains the measured pulse widths — and on its own it cannot separate a rate from **double** that rate. Every pulse that is a whole number of bit times is also a whole number of *half* bit times, so both readings fit equally well: measured 0.99999999 for each, on the same capture. The halved reading then finds twice as many frames, which any error count rewards.

So a shorter bit time is rejected on separate evidence, by two tests that divide the work between them. Both are consulted only when the measurement itself lands on a standard rate; where it does not, a rescaling is the only route to the truth and neither test runs.

- **Some pulse must span an odd number of the candidate's bit cells.** Real traffic has single- and three-bit runs, and at half the true rate every pulse spans an even count. Only runs up to 12 bit times count, because a longer one is inter-byte idle whose length is set by when the capture started rather than by the format. Measured over 368 640 decodes spanning every capture start position: at a halved bit time no start position misreads the rate.
- **The short end of the widths must not span more than ~2.5 of the candidate's bit cells.** The test above is arithmetically confined to even divisors — halving, quartering and sixthing a bit time make every count even, while dividing by an odd number leaves the counts' parity exactly as it was, so thirds and fifths pass it whatever the payload. What separates those is the minimum: ordinary traffic has single-bit runs, since a ten-bit frame opens with a one-bit start bit. Measured on one capture, the short end against the candidate's own bit time: **1.00** at the truth, 2.00 at a half, 3.00 at a third, 5.00 at a fifth. **The bound must sit below 3**, because a 3x misfit lands at exactly 3.00 and that is the largest family — measured on the LIN-break vector at 19 200, 57 600 and 76 800 Bd over 60 capture openings each, 2.0, 2.5 and 2.9 report the true rate every time while 3.0 and 3.5 give back 42, 60 and 60 wrong answers. 2.5 is the middle of that range, so neither edge is load-bearing. **What it costs:** a payload whose shortest genuine run is three bit times or more — `0x00` with a two-bit gap — cannot have a third-of-the-rate reading admitted once the fit snaps; measured, that capture reports a third of its true rate with the test and without it alike. The **5th percentile** of the widths is used rather than the outright minimum, since one narrow glitch would otherwise silence it — and such a glitch is itself a route to a fifth-of-the-bit-time reading.

Samples per bit decides how many candidates are admissible at all, which is why the rate a capture is digitised at matters as much as the waveform. A reading is refused outright below 4 samples per bit, so at the ~8 samples/bit a locked rate gives, only the halved candidate clears that floor; at the 1 MS/s the first probe of an unlocked capture uses, a 31 250 Bd line arrives at 32 samples per bit and every divisor from 2 to 6 clears it. The probe pass is therefore where a misfit is decided, and it keeps its own answer whenever no lower listed rate would oversample adequately.

Neither test touches the *longer*-bit-time direction, so the genuine ambiguity below is preserved, and the note row still names a rival rate when one really fits.

Perfectly periodic traffic can be genuinely undecidable: eight `0x00` frames at 9600 Bd 7N1 with a one-bit gap are bit-for-bit the same waveform as four `0x08` bytes at 4800 Bd 8N1. No decoder can tell which device sent it, so the app says so instead of guessing. Irregular gaps between bytes remove the ambiguity entirely.

A capture beginning mid-byte costs **2 to 12 bytes**, measured across thirty baud rates, and does not scale with the rate — it is at most one frame plus the trigger's own offset. Those bytes are displayed but their bit boundaries are wrong, so they are arbitrary groupings of real wire bits.

Data Bits — why `Auto (any)` is not the default

`Auto (7/8)` searches 7 and 8 data bits. `Auto (any)` also tries 5, 6 and 9. Measured over 105 hostile-signal cases, the wider search changed the answer in **5** of them, and only **1** was a genuine 9-bit stream: the other 4 were damaged captures that a wider frame made look *cleaner* while being wrong, because the extra data cell absorbs whatever broke the stop bit. 9 data bits is searched **only** when explicitly forced, for the same reason.

Only one stop bit is searched and reported: a second stop bit is indistinguishable from a bit time of idle, which the decoder already tolerates. A 2-stop line decodes correctly and reports 1 stop.

Auto-lock conditions

Auto-lock is on by default and conditional. It locks only when **all** of these hold:

- the rate is above the 100 Bd floor and below the 250 kBd ceiling
- at least 8 good frames, and a clean unbroken run covering ~60 % of the capture
- `FIT` at 0.9 or better
- the logic levels were stable — no drifting or noisy baseline

A non-standard rate locks too, and that is the point. The first condition asks only whether the figure is a rate at all; landing on the standard ladder is not required, so a device on an awkward divisor is lockable and its recording modes work without a hand-typed number. The three conditions after it are what judge the evidence — snapping would judge only tidiness.

What gets locked is rounded, not the raw measurement: the coarsest step that moves the rate under 1 % — nearest 100 Bd above 10 kBd, else nearest 10, else whole baud. So a device set to 16100 Bd measuring 16099 locks as **16100**, while 105 Bd locks as 105 rather than being dragged to 110. A rate already on the standard ladder is never moved, which is what keeps MIDI's 31250 from becoming 31300.

Otherwise the padlock stays amber and **Lock Rate** is offered. Polarity is **never** locked; it is re-derived every capture, because freezing it off one short detect capture is how a confidently inverted, entirely wrong decode happens.

Flow control

`Options ▸ Rear BNC = FC Out` makes the rear EXT TRIG OUT emit **one** pulse once the digitizer is armed: idle-low at **+0.20 V, +4.45 V high, 5.72 µs wide, 436 ns rise** on a scope — the same figures given above, not the superseded 75 Ω-cabling ones. The width is the firmware's own and cannot be set here, so the receiving device needs an edge-triggered input rather than a polling loop. It is a **credit, not CTS**: treat the pulse as permission to send no more than one capture can hold. It does nothing unless the device is built to wait for it.

One press then runs the whole conversation, one credit per window, until a credited window comes back silent or the 32-window bound fires. The device has to go quiet when it has nothing to send: that silence is the only signal the app has that the transmission is over.

Triggering, in both directions: what is tested

The bench wires both directions permanently — SDG CH2 into the DMM's rear TRIGGER IN, and the DMM's rear TRIGGER OUT into the generator's rear In/Out — with every line also on a scope channel. What follows separates what has been **measured** from what has not, in each direction.

Into the DMM: an external pulse starts a decode — WORKS

the marker, measured on the scope	**0 to +4.972 V**, 50.00 µs wide, **100 ns** rise, +8 mV overshoot
against the DMM's EXT TRIG IN spec	≥ 1 µs wide, 0–5 V TTL — **50× the width floor, 3 V of level margin**
pulse present, quiet line	capture completes in **1.1 s**
pulse absent, quiet line	**refuses after the full wait**, `edge trigger unavailable; captured free-running`

The negative is the evidence. `acq_triggered()` falls back to a free-running capture when no trigger arrives, and a free-run on a quiet line still returns samples — so "it completed" proves nothing on its own. What proves it is that removing the pulse changes the outcome *and* names the reason.

`Rear BNC` alone is not enough for an anchored capture, and `sdec.trigext_only` is why. `trigext` **OR**s the rear input with the analog start-bit trigger — the right default, because "start on a start bit, or when the DUT asserts its GPIO, whichever comes first" is not expressible as a single choice. But on a line that never stops transmitting the start bit always wins, so the external pulse cannot decide *where* the window opens. Measured on a live line:

mode	marker	result
`trigext` (OR)	off	completes in **1.1 s**, `trigblended = true` — the start bit won
`trigext_only`	off	**times out at 6 s**, `trigblended = false` — the busy line is ignored

mode	marker	result
`trigext_only`	on	completes in **1.2 s** — the marker armed it

The middle row is the one that cannot be faked: without exclusivity a busy line completes every capture. `sdec.trigext_only` is **off by default** and has no panel control — it makes a capture depend on equipment the product does not require, and a user with one probe must never wait on a pulse that cannot arrive.

So `Rear BNC` means something different in each of the three Trigger modes, and only two of them are useful:

`Options ▸ Trigger`	what the rear input does	how
`Start bit`	**OR'd** with the analog start-bit trigger	`blender[1]` over `EVENT_ANALOGTRIGGER` + `EVENT_EXTERNAL`, `orenable = true`
`Trigger key`	**OR'd** with the front key — the only competing source is the operator, so this is the panel-reachable way to make the pulse matter on a busy line	same blender, `EVENT_DISPLAY` + `EVENT_EXTERNAL`
`Free run`	**ignored entirely**	the blender branch is guarded `sdec.trigmode ~= 'free'`, and free run never reaches `acq_triggered()` at all

The Free-run case is reported, and by the one mechanism that cannot be reached by the wrong path. `sdec.options_apply()` assigns `sdec.trigext` without a note of its own, and the one-shot lognote in `sdec.trigext_toggle()` fires only if that speculatively-wired status-row rect ever delivers a press — so neither is what carries the warning. `sdec.ui_notes()` **derives it from state instead of announcing it on change**: while the combination holds, every refresh re-adds `Rear BNC = Trig In is set but IGNORED -- Free run means do not wait for anything`, and `ui_trig_t` appends a `?` to `EXT TRIG IN`. A recomputed note cannot be missed by arriving through a path nobody thought to instrument, which is why the setting-looks-applied-but-is-not class of warning belongs there and not in a handler.

When the blender is unavailable the code falls back to the rear BNC **alone** and says so (`no trigger blender; using the rear BNC alone`) rather than dropping the operator's request. Four-case regression proof: `tools/bench_trigin.py`, to be re-run after any change to `acq_triggered()`.

What is NOT yet shown is anchoring to a payload offset. That needs the marker phase-locked to the signal arb, which needs a marker waveform uploaded alongside it. Proven here is that an external pulse, and only that pulse, starts the capture.

Out of the DMM: a credit makes the generator send more — NOT YET

Three gates, in order, and it fails at the third:

gate	result
shape of the credit pulse	**PASS** — idle-low, +4.45 V, 5.72 µs, 436 ns rise, square corners (above)
does burst keep TrueArb?	**PASS** — `BTWV STATE,ON` leaves `SRATE MODE,TARB,VALUE,96000Sa/s` intact, and `STPS,0` needs no change. This was the make-or-break: had burst forced DDS, every stored vector would be resampled and interpolated, and none of them would be valid stimulus any more

gate	result
does the generator act on the pulse?	**FAIL** — with `TRSR,EXT`, `GATE_NCYC,NCYC`, `TIME,1` and the output on, the generator stayed silent through a credit

The negative control holds: armed and uncredited, the generator was silent for 2.5 s and the scope stayed `Ready`, so "no burst" is a real observation rather than a missed one.

What has not been separated is the burst machinery from the external input — the manual-trigger control that would distinguish them (`TRSR,MAN` + `MTRIG`) returned no status, so it is unresolved rather than answered. The leading suspect is **pulse width**: the generator's minimum trigger width is nowhere specified, and 5.7 µs is the narrowest thing this bench can present. Widening it to ~100 µs is the next test, and it needs a one-shot on the credit line rather than any change to the app, since `trigger.extout.pulsewidth` cannot be set.

So the README's standing claim is unchanged and still correct: flow control is **verified electrically but never closed as a loop against a device that waits for it.** What is new is that the reason is now located in the generator's trigger input rather than unknown.

Endurance: what the soaks measured

Two 17-hour soaks, each **6 laps of the full 1 683-point matrix, 10 098 capture-and-decode cycles on one power cycle**, agreeing within **0.2 % on duration and 1.6 % on failure count** — 318 failures against 313. The headline is in the README; this is the shape of it.

Lap time is the leak indicator, because a leaked display object, an unreturned buffer handle or a fragmenting allocator wedges the instrument around hour six without failing a single point. It fell in both runs. The second's six laps ran **10 567, 10 219, 10 254, 10 282, 10 173 and 10 299 s**, each within ±1 % of its counterpart in the first.

3.1 % of points fail, against a matrix built to attack the decoder — LIN traffic presented to a UART decoder, single-byte and walking-bit patterns, sinusoidal drift, and swings and offsets swept to the edge of range. It is not comparable to the rate-detection figures under **Known failures of automatic rate detection** in the manual. Two properties:

- **It concentrates.** Four vectors — `v94`, `v63`, `v90`, `v71` — account for **76 %** of all failures; `v63` is a LIN frame whose 13-bit break field is not valid 8N1.
- **It is mostly intermittent.** The 318 failures are **162 distinct cells**, of which **88 failed in exactly one lap of the six** and **6 in all six**.

Cross-run agreement is stronger than within-run agreement. Comparing which fixed-stimulus cells failed, a lap of the second run differs from its counterpart in the first by **11–16 cells**, against **21–32** for every pair of laps inside the first run. The failure set is a property of the matrix, not of the run — which is what licenses reading a cell that fails in all six laps as a property of the cell.

A third, 7-hour soak covers the **recording** path, which neither long soak exercises: **81 laps, 3 726 cycles and 8 one-press recordings, zero failures**, every recording ending on its byte ceiling.

Not covered by any soak: many power cycles rather than one, and front-panel interaction — that is `hw-panel`'s job, under **The release gate, stage by stage** in BENCH.md.

Firmware limits worth knowing

One app build per power cycle. `sdec.start()` builds the display objects and the firmware does not appear to return the pool fully, so after enough rebuilds in one power cycle the instrument reports "*the maximum number of objects have already been created*" and the app refuses to build rather than coming up with controls missing. How many is enough is **not characterised** — it has been seen after a few dozen. One relaunch is safe; a long editing session needs a power cycle. This affects development, not normal use.

The build itself is **122 display objects** live after `ui_build()`, and **134** once the options screen has been built too. Counted with the harness census, not estimated.

The panel geometry is baked in rather than negotiated at runtime, and it does not need to be: the panel is pixel-identical across the entire TSP range. Object `y` is relative to a content area **49 px below the panel top**, and a screen title is limited to **31 characters**. `serial_ui.tsp` is 1 300 lines built against those constants, and they carry to another TSP model unmodified — so a port is an acquisition question only, never a layout one.

The display-side unknown is not geometry but the **object pool** above: an allocation limit, not a dimension, and uncharacterised on every model including this one.

Event 4915 — "attempting to store past the capacity of a reading buffer" — is ERROR severity, which the front panel shows as a **modal dialog over the app** whatever `localnode.showevents` says. Two defences apply to every armed capture: `fillmode = 1` per capture (the numeric value; `buffer.FILL_CONTINUOUS` does not exist on this firmware and assigning it raises), and 100 readings of capacity past the count the trigger model is asked for. Measured with no headroom: 20–30 events per capture.

`localnode.showevents` governs the remote interface only. It cannot suppress the panel's dialog, so muting is not a defence — not posting the event is.